

使用曦云 MetaX C500 运行 ERNIE-Image 模型

本文档整理了 ERNIE-Image 在曦云 MetaX C500 环境下的适配过程、部署方式、服务化方案、Web UI 方案，以及已完成的一轮性能测试结果。

当前结论可表述为：

ERNIE-Image 已在 MetaX C500 环境完成初步功能适配验证，模型主推理链路已跑通，并已验证单机单卡场景下的图像生成、API 服务、Web UI 和基础性能测试能力。当前结论属于功能级适配成功 / POC 跑通，尚不等同于“正式生产级适配完成”。

参考资料

- FastDeploy 参考格式文档：

https://github.com/PaddlePaddle/FastDeploy/blob/develop/docs/zh/get_started/installation/metax_gpu.md

- ERNIE-Image 开发分支：

<https://github.com/HsiaWinter/diffusers>

- Diffusers 官方合入 PR：

<https://github.com/huggingface/diffusers/pull/13432>

1. 概述

ERNIE-Image 已在曦云 MetaX C500 环境完成适配验证。本文档给出：

- 环境要求
- 模型与代码准备
- 安装与部署方法
- 单次推理验证
- API 服务与 Web UI 部署
- 常见问题
- 性能测试结果与专业评估

ERNIE-Image 当前采用基于 [diffusers](#) 的推理方案，结合 MetaX C500 的 GPU 环境完成图像生成任务迁移。实际部署中，无需改动模型权重格式，主要工作在于适配 Python 推理栈和 [transformers/diffusers](#) 版本组合。

2. 环境准备

2.1 基础要求

- Python ≥ 3.10
- Linux x86_64
- MetaX C500
- 建议使用 Conda 独立环境
- 建议提前准备：

- 模型权重包 [ERNIE-Image.tar](#)
- 可访问 GitHub 的网络
- [/data](#) 目录用于模型、日志与输出存储

2.2 已验证环境

项目	版本 / 信息
Chip Type	MetaX C500
Python	3.10.10
Torch	2.6.0+metax3.3.0.2
Diffusers	0.38.0.dev0 (HsiaWinter/diffusers add-ernie-image)
Transformers	5.6.0.dev0
Gradio	6.11.0
MACA	3.3.0.15
Driver	3.6.11
BIOS	1.27.5.0

说明：本次适配是在 MetaX C500 实例内完成，实际容器环境中可见 [MetaX C500 GPU](#)，推理已成功生成图片，并通过 API 与 Web UI 验证。

3. 模型与代码准备

3.1 模型权重

将 ERNIE-Image 模型权重包放置到服务器，例如：

```
/data/ERNIE-Image.tar
```

若文件名包含带鉴权参数的下载名，也可以直接使用，无需重命名。

3.2 推理代码

ERNIE-Image 当前使用自定义 [diffusers](#) 分支：

```
git clone https://github.com/HsiaWinter/diffusers
cd diffusers
git checkout add-ernie-image
```

参考来源：

- 开发分支： [HsiaWinter/diffusers add-ernie-image](#)
- 官方 PR： [huggingface/diffusers#13432](#)

4. 环境安装

建议使用独立 Conda 环境，避免污染现有业务环境。

4.1 创建环境

```
conda create -n ernie-image python=3.10 -y
conda activate ernie-image
```

4.2 安装基础依赖

```
pip install torch torchvision accelerate Pillow fastapi uvicorn gradio
```

4.3 安装 ERNIE-Image 对应 diffusers

```
git clone https://github.com/HsiaWinter/diffusers /data/ernie-image/src/diffusers
cd /data/ernie-image/src/diffusers
git checkout add-ernie-image
pip install -e .
```

4.4 安装兼容版 transformers

ERNIE-Image 权重中的 `text_encoder` 和 `pe` 配置包含 `mistral3/ministral3`。低版本 `transformers` 无法识别，会报：

```
KeyError: 'ministral3'
```

因此建议直接安装新版 `transformers`：

```
pip install -U "git+https://github.com/huggingface/transformers"
```

5. 模型解压与目录组织

建议目录结构如下：

```
/data/ernie-image/
├── model/
│   └── ERNIE-Image/
├── outputs/
├── logs/
├── src/
│   ├── diffusers/
├── benchmarks/
│   ├── run_benchmarks.py
│   └── results/
├── 01_single_run.py
├── 02_api.py
├── 03_start.sh
└── 04_selftest.sh
```

解压模型：

```
mkdir -p /data/ernie-image/model
tar -xf /data/ERNIE-Image.tar -C /data/ernie-image/model
```

若源文件名带 `?authorization=...`，也可直接：

```
tar -xf "/data/ERNIE-Image.tar?authorization=..." -C /data/ernie-image/model
```

解压后模型目录应类似：

```
ERNIE-Image/  
├── model_index.json  
├── transformer/  
├── vae/  
├── scheduler/  
├── tokenizer/  
├── text_encoder/  
└── pe/
```

6. 环境验证

安装完成后，可通过以下代码验证环境：

```
import torch  
from diffusers import ErnieImagePipeline  
  
print("torch:", torch.__version__)  
print("cuda available:", torch.cuda.is_available())  
print("device count:", torch.cuda.device_count())  
if torch.cuda.is_available():  
    print("device name:", torch.cuda.get_device_name(0))  
  
print("pipeline:", ErnieImagePipeline)
```

如果以上代码执行成功，则说明基础环境已可用于 ERNIE-Image 推理。

7. 单次推理验证

建议先做单次推理，再启动服务。

7.1 推理示例

```

import os
import random
import numpy as np
import torch
from diffusers import ErnieImagePipeline

os.environ["CUBLAS_WORKSPACE_CONFIG"] = ":4096:8"
os.environ["HF_HOME"] = "/data/huggingface_home"

seed = random.randint(0, 100000)
random.seed(seed)
np.random.seed(seed)
torch.manual_seed(seed)
torch.cuda.manual_seed_all(seed)
torch.backends.cudnn.deterministic = True
torch.use_deterministic_algorithms(True)
torch.backends.cudnn.benchmark = False

pipe = ErnieImagePipeline.from_pretrained(
    "/data/ernie-image/model/ERNIE-Image",
    torch_dtype=torch.bfloat16,
    trust_remote_code=True,
)
pipe = pipe.to("cuda")
pipe.transformer.eval()
pipe.vae.eval()
pipe.text_encoder.eval()
pipe.pe.eval()

generator = torch.Generator(device="cuda").manual_seed(seed)

output = pipe(
    prompt="一朵鲜艳的红色玫瑰，棚拍，白色背景",
    height=512,
    width=512,
    num_inference_steps=4,
    guidance_scale=5.0,
    generator=generator,
    use_pe=False,
)

output.images[0].save("/data/ernie-image/outputs/single-run.png")
print("done")

```

7.2 输出检查

```
ls -lh /data/ernie-image/outputs/single-run.png
```

本次实际验证结果：

- 输出文件存在
- 文件格式：PNG
- 分辨率：512x512
- 文件大小：约 225 KB

8. 服务化部署

8.1 启动 API 服务

建议基于 FastAPI + Uvicorn：

```
source /opt/conda/etc/profile.d/conda.sh
conda activate ernie-image

export ERNIE_IMAGE_MODEL_DIR=/data/ernie-image/model/ERNIE-Image
export ERNIE_IMAGE_OUTPUT_DIR=/data/ernie-image/outputs
export HF_HOME=/data/huggingface_home

python /data/ernie-image/02_api.py
```

默认监听:

```
0.0.0.0:8188
```

8.2 健康检查

```
curl http://127.0.0.1:8188/health
```

返回示例:

```
{
  "status": "ok",
  "model_dir": "/data/ernie-image/model/ERNIE-Image",
  "cuda_available": true,
  "device_count": 1
}
```

8.3 生成接口

```
curl -X POST http://127.0.0.1:8188/generate \
-H "Content-Type: application/json" \
-d '{
  "prompt": "一只黑白相间的中华田园犬",
  "height": 512,
  "width": 512,
  "num_inference_steps": 4,
  "guidance_scale": 5.0,
  "use_pe": false
}'
```

返回示例:

```
{
  "status": "ok",
  "output_path": "/data/ernie-image/outputs/1775653895-0f4e5455.png",
  "download_url": "/files/1775653895-0f4e5455.png",
  "revised_prompts": null,
  "elapsed_sec": 1.64,
  "seed": 38854
}
```

8.4 文件下载

```
curl http://127.0.0.1:8188/files/1775653895-0f4e5455.png --output result.png
```

9. Web UI 部署

为便于交互式体验,可在同一服务中挂载 Gradio Web UI。

9.1 访问地址

```
http://127.0.0.1:8188/ui
```

根路径会自动跳转：

```
GET / -> /ui
```

9.2 UI 功能

- Prompt 输入
- Height / Width 调整
- Steps / Guidance 调整
- Seed 输入
- `use_pe` 开关
- 示例 Prompt
- 图片预览
- 生成元信息展示

9.3 SSH 隧道访问

如服务部署在远端实例，可通过隧道访问：

```
ssh -CNg -L 8188:127.0.0.1:8188 root@<host> -p <port>
```

然后本地浏览器访问：

```
http://127.0.0.1:8188/ui
```

10. 性能测试

10.1 测试说明

本轮性能测试已在当前 MetaX C500 实例上完成，测试内容涵盖：

- 吞吐测试
- 时延测试
- 显存峰值测试
- 512 / 1024 / 不同步数性能曲线
- 单并发 / 多并发性能对比
- 与 CUDA 基线对比

测试对象：

- 当前已部署 FastAPI 服务：<http://127.0.0.1:8188>

采样方式：

- API 端到端 wall time
- `mx-smi` 采集 GPU 利用率、功耗、温度、显存占用

测试脚本：

- [/data/ernie-image/benchmarks/run_benchmarks.py](#)

测试报告：

- [/data/ernie-image/benchmarks/results/benchmark_report_20260408T135747Z.json](#)

说明：

- 本轮测试是在当前服务实现上完成的
- 当前服务使用了进程级串行锁，因此多并发测试反映的是现有服务端到端行为，不是 GPU 真并行极限

10.2 单图性能矩阵

分辨率	Steps	时延(s)	吞吐(img/s)	峰值显存(MiB)	进程显存峰值(MiB)	峰值 GPU 利用率	平均 GPU 利用率	峰值功耗(W)	峰值温度(C)
512	4	1.632	0.6128	36536	35710	85	77.75	234	44
512	8	2.902	0.3446	36536	35710	89	71.83	241	45
512	20	6.783	0.1474	36536	35710	89	81.54	248	48
512	50	16.566	0.0604	36536	35710	89	88.03	244	53
1024	4	6.135	0.1630	36536	35710	97	89.17	270	54
1024	8	11.577	0.0864	36536	35710	97	89.59	271	56
1024	20	27.971	0.0358	36536	35710	97	92.63	273	59
1024	50	68.144	0.0147	36536	35710	97	94.62	276	61

10.3 单并发 / 多并发对比

测试配置：512x512, 8 steps, 共 8 个请求

并发	总耗时(s)	吞吐(req/s)	平均时延(s)	P50(s)	P95(s)	最大时延(s)	峰值显存(MiB)	峰值 GPU 利用率
1	23.058	0.3469	2.882	2.862	2.966	2.995	36536	89
2	23.269	0.3438	5.456	5.760	5.930	5.961	36536	89
4	23.163	0.3454	9.399	11.540	11.596	11.605	36536	89

10.4 结果分析

吞吐测试

- 当前部署下，512x512, 8 steps 的稳定吞吐约为 0.345 img/s
- 并发从 1 -> 4，吞吐基本不变，说明当前瓶颈不在客户端，而在服务端串行推理模型

时延测试

- 在 512 分辨率下，steps 增长与时延近似线性增长
- 在 1024 分辨率下，时延显著抬升
- 1024x50 已达到 68.144s/张

显存峰值测试

- 本轮测试中设备峰值显存约 36536 MiB

- Python 推理进程峰值显存约 35710 MiB
- 说明当前常驻服务 + 模型加载后的显存占用已经较高

512 / 1024 性能曲线

同 steps 下, 1024 相比 512 的时延倍率约为:

- 4 steps: $6.135 / 1.632 \approx 3.76x$
- 8 steps: $11.577 / 2.902 \approx 3.99x$
- 20 steps: $27.971 / 6.783 \approx 4.12x$
- 50 steps: $68.144 / 16.566 \approx 4.11x$

可认为:

1024 分辨率相对 512 分辨率, 时延成本大约增加 4 倍。

单并发 / 多并发对比

当前服务由于存在进程级串行锁:

- 并发升高时, 吞吐几乎不变
- 但请求排队导致平均时延、P50、P95 明显升高

因此当前实现更适合:

- 低并发
- 离线生成
- 单用户交互式 Web UI

不适合直接作为高并发在线图像生成服务。

10.5 与 CUDA 基线对比

本轮测试 未完成 CUDA 基线对比。

原因:

- 当前授权边界内只有 MetaX C500 实例
- 没有同模型、同参数、同软件栈的 NVIDIA/CUDA 基线机器

因此当前只能给出结论:

已完成 C500 侧绝对性能测试, 未完成与 CUDA 的相对对比测试。

如果要完成这部分, 需要额外准备:

- 一台 NVIDIA/CUDA 主机
- 同样的 ERNIE-Image 权重
- 同样的 prompt、分辨率、steps、服务实现
- 同样的测试脚本与采集方式

11. 性能测试结论

ERNIE-Image 已在 MetaX C500 环境完成初步适配验证, 模型主推理链路已跑通, 并已验证单机单卡场景下的图像生成、API 服务、Web UI 以及基础性能测试能力。当前结论可视为功能级适配成功, 后续仍需补充 CUDA 基线对比、并发服务架构优化、长稳测试与生产化验证, 才能认定为正式适配完成。

12. 常见问题

12.1 `KeyError: 'minstral3'`

原因：`transformers` 版本过低，不支持权重配置中的 `minstral3`。

解决方式：

```
pip install -U "git+https://github.com/huggingface/transformers"
```

12.2 `flash_attn` 导入失败

日志可能出现：

```
flash_attn is installed but failed to import ...  
Falling back to native PyTorch attention.
```

说明当前环境未启用对应 Flash Attention 实现，但不会阻塞推理，会自动回退到 PyTorch 原生 attention。

12.3 首次请求较慢

首次请求需要加载模型权重，耗时会明显高于后续请求，这是正常现象。建议：

- 服务启动后先做一次 warmup 请求
- 正式压测时区分冷启动和热启动性能

12.4 `use_pe=True` 的兼容性

PE 可用于 Prompt 增强，但可能：

- 增加推理链路复杂度
- 影响严格指令跟随
- 增加首轮加载开销

建议先使用：

```
use_pe=False
```

待主链路稳定后再逐步开启。